



ITSIMPLERA SOLUTION

Full Stack Web Development Internship Program

TECHNICAL DOCUMENTATION

Secure Multi-User Task Management API with JWT Authentication

Week 2 Task Submission Documentation

<i>Submitted By</i>	Abdul Haseeb
<i>Submitted To</i>	ITSimplera Solutions
<i>Project Title</i>	Secure Multi-User Task Management API with JWT Authentication
<i>Internship Track</i>	Full Stack Web Development
<i>Task Reference</i>	Week 2 Task — ITSimplera Solutions
<i>Repository</i>	GitHub Repo Link
<i>Date</i>	9 July 2026

Table of Contents

Table of Contents2

2. Executive Summary & Objective3

 2.1 Project Overview3

 2.2 Scope of the Solution3

 2.3 Business & Learning Objectives3

3. System Architecture & Directory Structure4

 3.1 Architectural Pattern: Express MVC4

 3.2 Directory Structure4

 3.3 File & Folder Responsibilities4

4. Database Schema Design (ERD & Model Details)5

 4.1 User Schema5

 4.2 Task Schema6

 4.3 Design Decisions7

 4.3.1 Virtual Properties — isOverdue7

 4.3.2 Compound and Text Indexes7

 4.3.3 Non-Destructive Deletion (Soft Delete)7

5. Security & Defensive Design Implementations8

 5.1 JWT Authentication & Middleware Protection8

 5.2 Password Safety8

 5.3 Request Validation8

 5.4 API Hardening8

 5.5 Data Isolation (Multi-Tenancy Guard)9

6. Advanced Query Features9

 6.1 Full-Text Search9

 6.2 Dynamic Filtering9

 6.3 Pagination9

 6.4 Flexible Sorting9

 6.5 Dashboard Statistics9

 6.6 Soft Delete9

7. API Endpoint Reference10

 7.1 Authentication Endpoints10

 7.2 User Profile Endpoints10

 7.3 Task Management Endpoints10

8. Postman Testing Guide & Verification10

 8.1 Step 1 — User Registration11

 8.2 Step 2 — Login & Bearer Token Setup12

 8.3 Step 3 — Task Insertion13

 8.4 Step 4 — Statistics & Query Parameters14

9. Centralized Error Handling15

 9.1 The AppError Class16

 9.2 The catchAsync Wrapper16

 9.3 Centralized Middleware — errorHandler.js16

Document Control17

2. Executive Summary & Objective

2.1 Project Overview

Developed by Abdul Haseeb for the ITSimplera Institute internship, this Node.js, Express, and MongoDB REST API enables secure multi-user task management. It structurally guarantees strict data isolation, preventing any user from accessing, modifying, or deleting another account's data.

Built for security, isolation, and robustness, the system integrates authentication, validation, and error handling directly into its middleware and schemas. This ensures all requests are authorized and sanitized before execution, while automatically converting any failures into structured JSON responses.

2.2 Scope of the Solution

The implementation addresses four functional pillars, each of which is expanded upon in subsequent sections of this document:

- **Secure Multi-User Data Isolation** — every task record is bound to an owning user ID at the database layer, and every controller query is scoped by the authenticated requester's identity, making cross-account data leakage structurally impossible rather than merely policy-enforced.
- **Advanced Task Operations** — full-text search, multi-field dynamic filtering, cursor-independent pagination, flexible multi-key sorting, and aggregation-driven dashboard statistics.
- **Defensive Security Measures** — JWT-based stateless authentication, bcrypt password hashing, HTTP header hardening via Helmet, brute-force mitigation via rate limiting, and NoSQL injection prevention via input sanitization.
- **Operational Reliability** — a centralized error-handling pipeline that normalizes Mongoose, JWT, and application-level errors into consistent, client-safe HTTP responses.

2.3 Business & Learning Objectives

From an academic standpoint, this task was designed to evaluate the intern's proficiency in constructing a layered Express.js architecture, applying industry-standard authentication patterns, and reasoning about database performance and security at a level consistent with professional backend engineering practice. The completed project satisfies these objectives and is version-controlled in the linked GitHub repository, reflecting a complete, reproducible, and independently verifiable body of work.

3. System Architecture & Directory Structure

3.1 Architectural Pattern: Express MVC

The application adapts a layered MVC architecture for a headless REST API, replacing server-rendered templates with JSON response payloads. This separation of concerns isolates database logic from request handling, keeps routing independent, and centralizes cross-cutting behaviors—like authentication and validation—into reusable middleware to eliminate code duplication.

Request flow is strictly unidirectional: incoming HTTP requests pass through global security and parsing middleware, advance through route-specific guards, and finally invoke the designated controller to interact with Mongoose models. Any error triggered at any layer is automatically forwarded via the next(error) mechanism straight to the centralized error handler.

3.2 Directory Structure

ITSimplera_Task2_Secure-Multi-User-Task-Management-API/

config/

db.js

MongoDB Atlas connection bootstrap

models/

User.js

User schema, password hashing hooks

Task.js

Task schema, virtuals, indexes

controllers/

authController.js

Register, login, logout logic

userController.js

Profile retrieval, password change

taskController.js

CRUD, search, filter, stats

routes/

authRoutes.js

/api/auth/*

userRoutes.js

/api/users/*

taskRoutes.js

/api/tasks/*

middlewares/

authMiddleware.js

JWT verification & req.user injection

validationMiddleware.js

express-validator rule sets & handler

errorHandler.js

Centralized error normalization

utils/

AppError.js

Custom operational error class

catchAsync.js

Async controller error-forwarding wrapper

jwtHelper.js

Token signing / verification helpers

.env

Environment secrets (excluded from VCS)

server.js

Application entry point

package.json

Dependency manifest & scripts

README.md

Setup & usage instructions

3.3 File & Folder Responsibilities

Path	Responsibility
config/db.js	Establishes and exports the Mongoose connection to MongoDB Atlas, including connection-string handling and startup failure logging.
models/User.js	Defines the user schema, including credential fields, a pre-save bcrypt hashing hook, and instance methods for password comparison and password-change tracking.
models/Task.js	Defines the task schema, virtual properties, compound and text indexes, and the soft-delete flags.

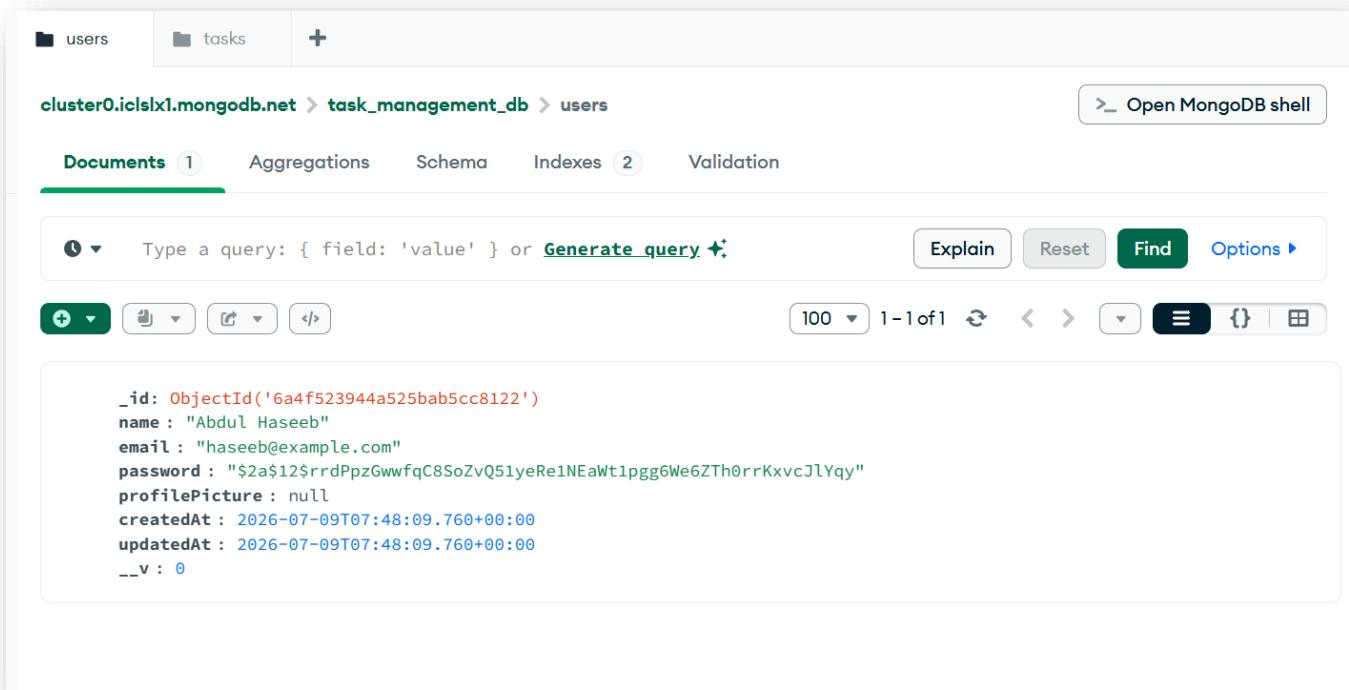
Path	Responsibility
controllers/authController.js	Implements registration, login, and logout handlers, and issues signed JWTs on successful authentication.
controllers/userController.js	Implements authenticated profile retrieval and password-change workflows.
controllers/taskController.js	Implements task CRUD, full-text search, filtering, pagination, sorting, and the statistics aggregation endpoint.
routes/*.js	Declaratively binds HTTP verbs and paths to controller functions, attaching authentication guards and validation chains per route.
middlewares/authMiddleware.js	Extracts and verifies the Bearer JWT, re-hydrates the requesting user from the database, and rejects stale tokens issued before a password change.
middlewares/validationMiddleware.js	Houses express-validator rule chains per endpoint and a shared handler that short-circuits requests with malformed input.
middlewares/errorHandler.js	Final Express error-handling middleware; normalizes all thrown/forwarded errors into a consistent JSON error contract.
utils/AppError.js	A custom Error subclass carrying an HTTP status code and an 'isOperational' flag used to distinguish expected failures from programming defects.
utils/catchAsync.js	A higher-order function that wraps async controller handlers so rejected promises are automatically forwarded to Express error handling, eliminating repetitive try/catch blocks.
utils/jwtHelper.js	Centralizes JWT signing (with expiry and secret configuration) and verification logic used across the auth middleware and controllers.
server.js	Application bootstrap: loads environment variables, initializes the Express app, mounts global middleware and routers, and starts the HTTP listener.

4. Database Schema Design (ERD & Model Details)

The data layer is modeled with Mongoose schemas that enforce structural and semantic constraints at the application boundary, complementing MongoDB's inherently flexible document model with explicit validation, typed fields, and relational references implemented via ObjectId reference fields.

4.1 User Schema

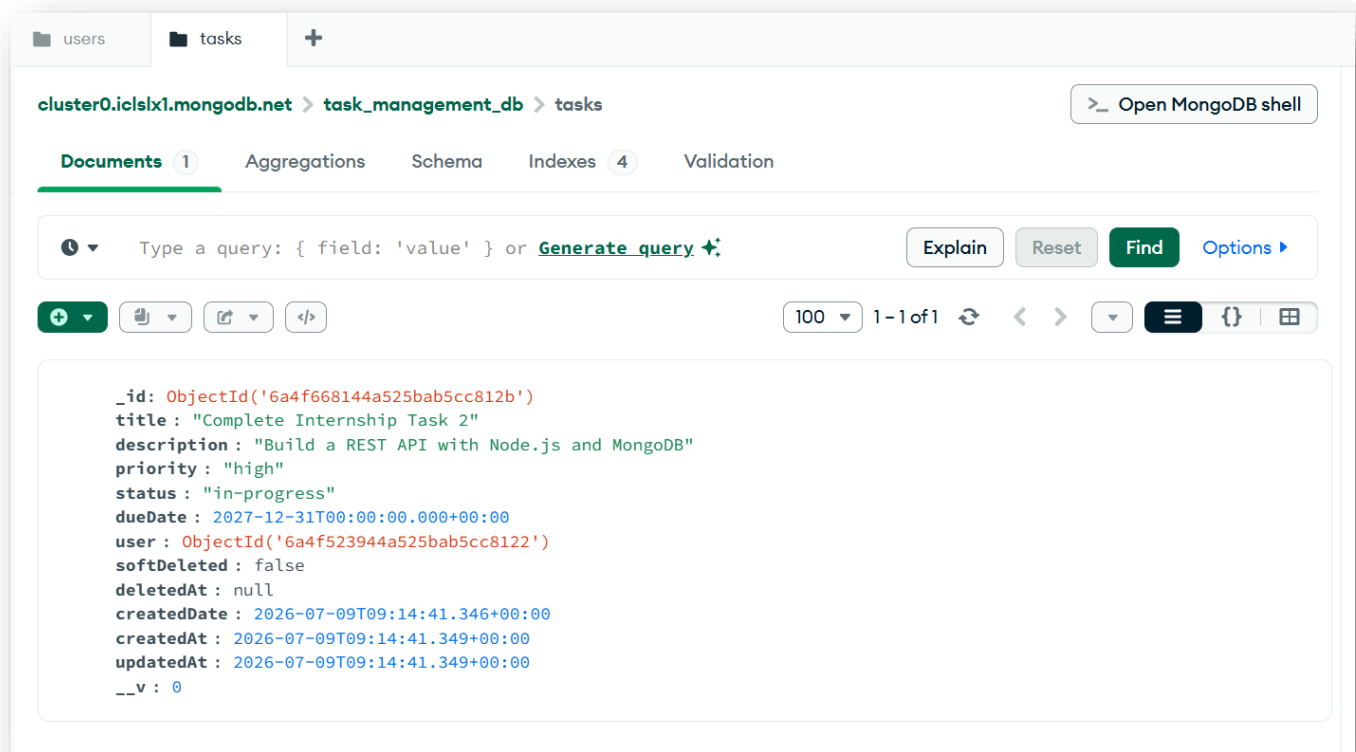
Field	Type	Constraints / Notes
name	String	Required; trimmed; maximum length enforced for display integrity.
email	String	Required; unique index; lowercased and trimmed; validated against a standard email pattern.
password	String	Required; minimum length 8; select: false so it is excluded from query results by default.
passwordChangedAt	Date	Set automatically whenever the password is updated; used to invalidate JWTs issued before the change.
createdAt / updatedAt	Date	Populated automatically via Mongoose's 'timestamps: true' schema option.



The User schema attaches a pre-save hook that intercepts document saves where the password field has been modified, hashing the plaintext value with bcryptjs before persistence, and a `changedPasswordAfter(jwtTimestamp)` instance method that compares the token's issued-at claim against `passwordChangedAt`, forming the basis of the token-invalidation logic described in Section 5.1.

4.2 Task Schema

Field	Type	Constraints / Notes
title	String	Required; trimmed; maximum length enforced; indexed for full-text search.
description	String	Optional; trimmed; indexed for full-text search.
status	String (enum)	One of pending, in-progress, completed; defaults to pending.
priority	String (enum)	One of low, medium, high; defaults to medium.
dueDate	Date	Optional; validated to ensure logical consistency with task lifecycle.
user	ObjectId (ref: User)	Required; foreign-key reference establishing task ownership; the cornerstone of data isolation.
softDeleted	Boolean	Defaults to false; flips to true on delete instead of removing the document.
deletedAt	Date	Populated at the moment of soft deletion; null otherwise.
createdAt / updatedAt	Date	Populated automatically via Mongoose's `timestamps: true` schema option.



4.3 Design Decisions

4.3.1 Virtual Properties — isOverdue

Rather than persisting an `isOverdue` boolean that would require background jobs to keep synchronized with the passage of time, the schema defines a Mongoose virtual property that computes overdue status on-the-fly by comparing `dueDate` against the current server time whenever the field is read, and only for tasks not already marked completed. This keeps the stored document minimal while still exposing a convenient, always-accurate field in the serialized API response, provided `toJSON` and `toObject` are configured with `virtuals: true`.

4.3.2 Compound and Text Indexes

Two categories of index were deliberately introduced to align with the query patterns exercised by the API's advanced features:

- A compound index on { user: 1, status: 1, priority: 1 } accelerates the overwhelming majority of task-list queries, which always filter by the owning user and frequently add status/priority filters on top. Because MongoDB can satisfy a query using a prefix of a compound index, this single index also serves user-only queries and user+status queries without needing separate indexes.
- A text index on { title: 'text', description: 'text' } enables the \$text query operator used by the full-text search feature (Section 6.1), allowing MongoDB to perform tokenized, case-insensitive matching internally rather than requiring inefficient regex scans across the collection.

4.3.3 Non-Destructive Deletion (Soft Delete)

Deleting a task sets `softDeleted: true` and stamps `deletedAt` with the current timestamp rather than issuing a MongoDB `deleteOne` call. All read queries in the controller layer therefore include a `softDeleted: false` (or equivalent \$ne)

predicate by default. This design preserves an audit trail for compliance and debugging purposes, allows for a future 'restore' capability with no additional schema work, and avoids the risk of irreversible data loss from an accidental or malicious delete request.

5. Security & Defensive Design Implementations

Security is treated as a first-class architectural concern rather than a peripheral add-on. This section details each defensive layer, from cryptographic password handling through to network-level abuse prevention.

5.1 JWT Authentication & Middleware Protection

- **Stateless JWTs:** Upon login, the server issues a signed JWT containing the user's ID with a set expiration window.
- **4-Stage Verification Middleware:**
 - **Extraction:** Retrieves the Bearer token from the Authorization header (rejects missing/malformed with a 401).
 - **Signature Check:** Validates the signature and expiration against the secret key (rejects forged/expired tokens).
 - **User Existence:** Re-fetches the user from the database to ensure the account still exists.
 - **Stale Token Check:** Rejects the token if the user changed their password after the token was issued.
 - **Controller Trust:** Passing all checks attaches the authenticated user to req.user, allowing downstream logic to safely trust the identity.

5.2 Password Safety

- **Password Hashing:** Plaintext passwords are never saved; a Mongoose pre-save hook automatically hashes new or modified passwords using bcryptjs with 12 salt rounds to resist brute-force attacks.
- **Leak Prevention:** The password field is configured with select: false, omitting it from standard query results to prevent accidental serialization into API responses unless explicitly requested via .select('+password').

5.3 Request Validation

Every mutating endpoint uses an express-validator rule chain to assert type correctness and enforce field constraints, such as a minimum 8-character password with mixed cases and numbers. If any payload is unexpected or malformed, a shared handler immediately short-circuits the request with a 422 Unprocessable Entity response containing specific field errors. This validation barrier completely ensures that invalid data never reaches the controllers or the database layer.

5.4 API Hardening

Mechanism	Package	Protection Provided
Secure HTTP Headers	helmet	Applies a curated set of HTTP response headers (Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security, and others) to mitigate clickjacking, MIME-sniffing, and related browser-level attack vectors.
Rate Limiting	express-rate-limit	Caps each client IP address to 100 requests per 15-minute window, throttling brute-force credential-stuffing attempts and general API abuse without materially affecting legitimate usage patterns.
NoSQL Injection Prevention	express-mongo-sanitize	Strips request keys containing Mongo operator characters (\$ and .) from req.body, req.query, and req.params before they can be interpreted as query operators, preventing operator-injection attacks such as { "email": { "\$ne": null } }.

5.5 Data Isolation (Multi-Tenancy Guard)

Data isolation is structurally embedded into the data-access layer rather than managed through optional filters. Every query issued by `taskController.js` enforces `user: req.user.id` as a mandatory predicate, sourcing the identity exclusively from the verified JWT payload rather than client-supplied inputs. Consequently, malicious requests attempting to access another user's task ID in the URL fail and return zero matching documents, making cross-account data leaks structurally impossible.

6. Advanced Query Features

Beyond basic CRUD, the Task API exposes a set of query capabilities designed to mirror the filtering, search, and reporting needs of a real production task manager.

6.1 Full-Text Search

A ``search`` query parameter is mapped to MongoDB's ``$text`` operator, which performs case-insensitive, stemmed, tokenized matching against the text index defined on ``title`` and ``description`` (Section 4.3.2). This is substantially more efficient than a ``$regex``-based approach, since ``$text`` queries can use the index directly rather than scanning every document, and MongoDB additionally returns a ``textScore`` metadata field that can be used to rank results by relevance.

6.2 Dynamic Filtering

The controller builds its Mongoose filter object incrementally: it always starts from the mandatory ``{ user: req.user.id, softDeleted: false }`` base, then conditionally appends ``status`` and ``priority`` predicates only when the corresponding query parameters are present in the request. This pattern avoids constructing a single rigid query shape and instead composes the exact filter required for each request, while guaranteeing the tenant-isolation and soft-delete predicates can never be omitted.

6.3 Pagination

List endpoints accept ``page`` and ``limit`` query parameters (with safe numeric defaults and upper bounds). These are translated into MongoDB's ``skip`` and ``limit`` cursor operations: ``skip = (page - 1) * limit``. To avoid the cost of an unbounded collection scan on every request, the total document count needed for pagination metadata (``totalItems``, ``totalPages``, ``hasNextPage``) is computed via a ``countDocuments`` call scoped to the same filter object used for the primary query, and both results are returned to the client in a structured ``meta`` object alongside the ``data`` array.

6.4 Flexible Sorting

A ``sort`` query parameter accepts one or more comma-separated field names, each optionally prefixed with a hyphen to indicate descending order (a convention consistent with popular REST API design practices such as JSON:API). The controller parses this parameter into a Mongoose-compatible sort object at request time, for example ``-priority,dueDate``, which sorts primarily by priority descending and secondarily by due date ascending, allowing clients to compose multi-key sort orders without any server-side code changes.

6.5 Dashboard Statistics

The `/api/tasks/stats` endpoint uses MongoDB's ``$facet`` stage to compute total tasks, status breakdowns, completion rates, and overdue metrics concurrently. By running multiple independent sub-pipelines inside a single query pass, it eliminates the need for separate database round trips. This approach drastically reduces network overhead and index reads while ensuring all statistics are calculated from a single, consistent snapshot of the data.

6.6 Soft Delete

As introduced in Section 4.3.3, the delete endpoint performs a targeted `updateOne` that sets `softDeleted: true` and `deletedAt: Date.now()` rather than a destructive `deleteOne`. Every read path filters on `softDeleted: false`, so soft-deleted tasks are transparently excluded from standard responses while remaining queryable directly (for compliance, audit, or administrative recovery purposes) by any process with direct database access.

7. API Endpoint Reference

7.1 Authentication Endpoints

Method	Endpoint	Body Fields	Auth Required	Success Status
POST	/api/auth/register	name, email, password	No	201 Created
POST	/api/auth/login	email, password	No	200 OK
POST	/api/auth/logout	— (Bearer token)	Yes	200 OK

7.2 User Profile Endpoints

Method	Endpoint	Body Fields	Auth Required	Success Status
GET	/api/users/profile	—	Yes	200 OK
PATCH	/api/users/change-password	currentPassword, newPassword	Yes	200 OK

7.3 Task Management Endpoints

Method	Endpoint	Body / Query Fields	Auth Required	Success Status
POST	/api/tasks	title, description, status, priority, dueDate	Yes	201 Created
GET	/api/tasks	search, status, priority, sort, page, limit (query)	Yes	200 OK
GET	/api/tasks/:id	—	Yes	200 OK
PATCH	/api/tasks/:id	any updatable task field	Yes	200 OK
DELETE	/api/tasks/:id	— (performs soft delete)	Yes	200 OK
GET	/api/tasks/stats	—	Yes	200 OK

Across all endpoints, failure conditions are mapped to standard HTTP semantics: 400 for malformed requests, 401 for missing or invalid authentication, 404 when a resource does not exist or is not owned by the requester, 409 for uniqueness conflicts such as duplicate email registration, and 422 for validation failures — the full mapping is detailed in Section 9.

8. Postman Testing Guide & Verification

The following sequence documents the manual verification workflow used to validate the API's authentication flow, validation rules, and advanced query features using Postman.

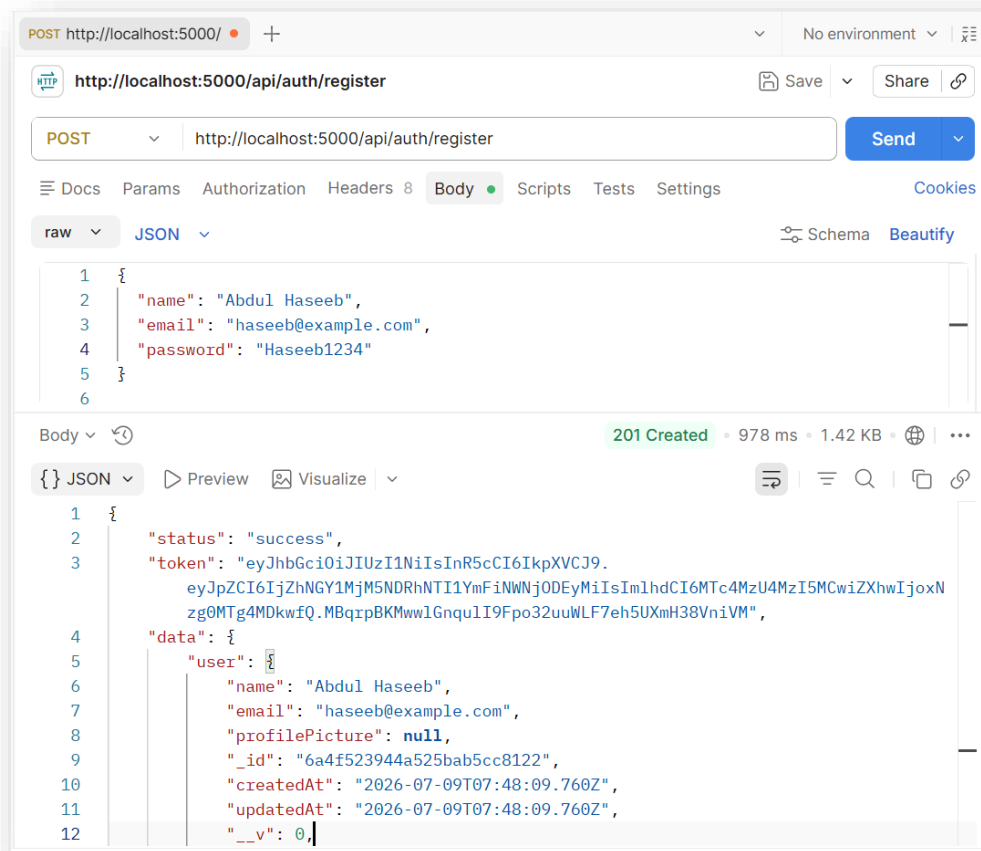
8.1 Step 1 — User Registration

A POST request is issued to `/api/auth/register` with a raw JSON body. The password validator requires at least one lowercase character, one uppercase character, one numeric digit, and a minimum length of 8 characters; requests that fail any of these criteria are rejected with a 422 response enumerating the specific violations.

```
POST /api/auth/register
Content-Type: application/json

{
  "name": "Abdul Haseeb",
  "email": "abdul.haseeb@example.com",
  "password": "Secure@123"
}

// 201 Created
{
  "status": "success",
  "data": {
    "user": {
      "id": "665f1c2a9b1e4a0012ab34cd",
      "name": "Abdul Haseeb",
      "email": "abdul.haseeb@example.com"
    }
  }
}
```



8.2 Step 2 — Login & Bearer Token Setup

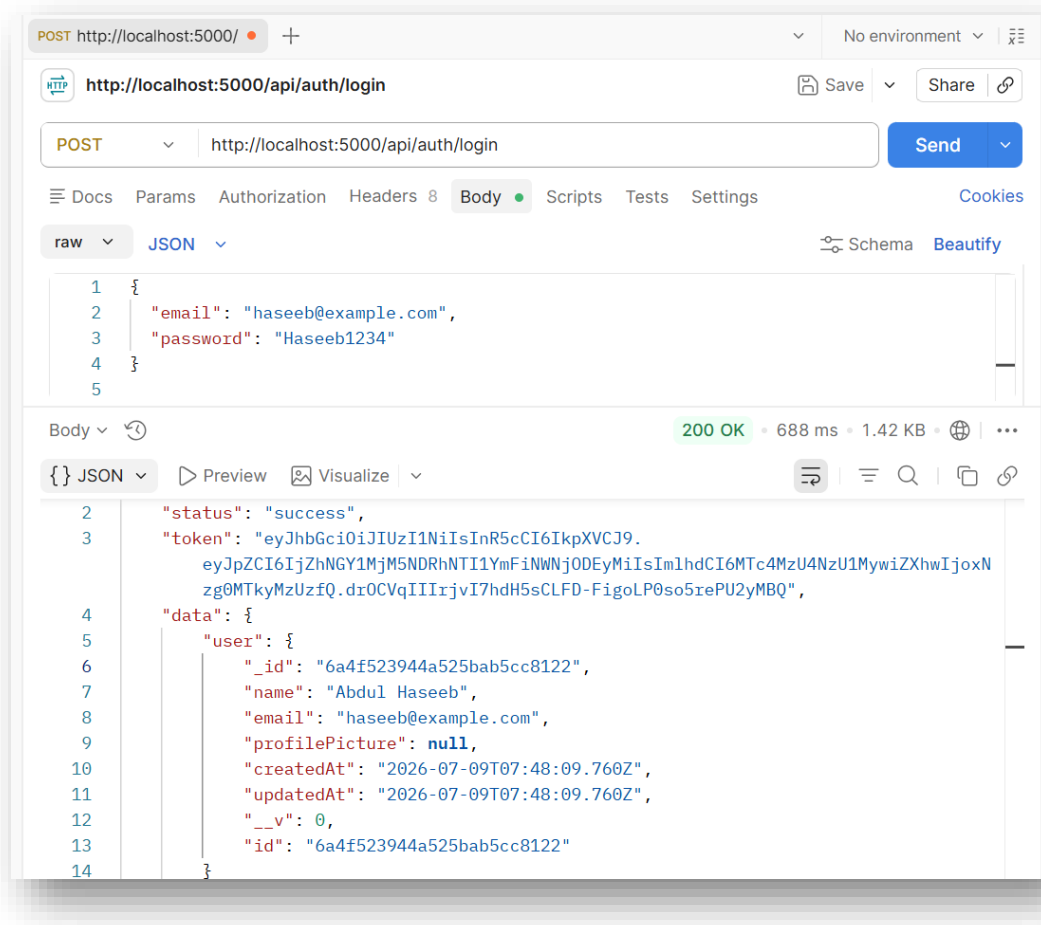
A POST request to `/api/auth/login` with valid credentials returns a signed JWT. In Postman, this token is copied into the collection or request-level Authorization tab, selecting the 'Bearer Token' auth type, so that it is automatically attached as an Authorization: Bearer <token> header on all subsequent requests within the collection.

```
POST /api/auth/login
Content-Type: application/json
```

```
{
  "email": "abdul.haseeb@example.com",
  "password": "Secure@123"
}
```

```
// 200 OK
```

```
{
  "status": "success",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "data": { "user": { "id": "665f1c2a9b1e4a0012ab34cd", "name": "Abdul Haseeb" } }
}
```



8.3 Step 3 — Task Insertion

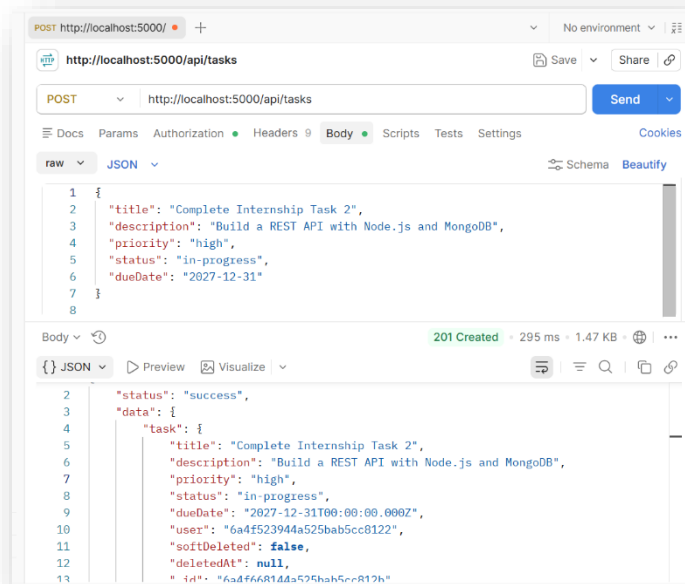
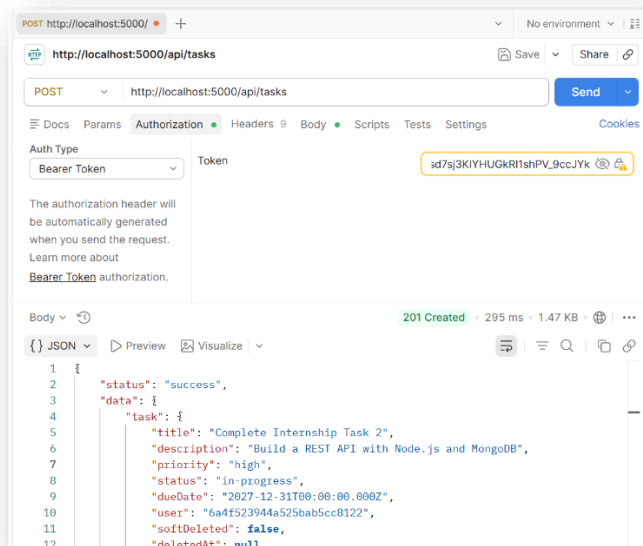
With the Bearer token applied, a POST request to `/api/tasks` creates a new task. The `dueDate` field is validated to ensure it represents a future timestamp relative to the server clock, preventing tasks from being created in an already-overdue state.

```
POST /api/tasks
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json
```

```
{
  "title": "Prepare Week 2 submission report",
  "description": "Compile documentation and push to GitHub",
  "priority": "high",
  "status": "in-progress",
  "dueDate": "2026-07-15T18:00:00.000Z"
}
```

```
// 201 Created
{
  "status": "success",
}
```

```
"data": {
  "task": {
    "id": "665f20aa9b1e4a0012ab3501",
    "title": "Prepare Week 2 submission report",
    "status": "in-progress",
    "priority": "high",
    "isOverdue": false
  }
}
```



8.4 Step 4 — Statistics & Query Parameters

A GET request to `/api/tasks/stats` returns the aggregated dashboard metrics described in Section 6.5. Separately, GET requests to `/api/tasks` accept combinations of query parameters to exercise search and filtering, for example:

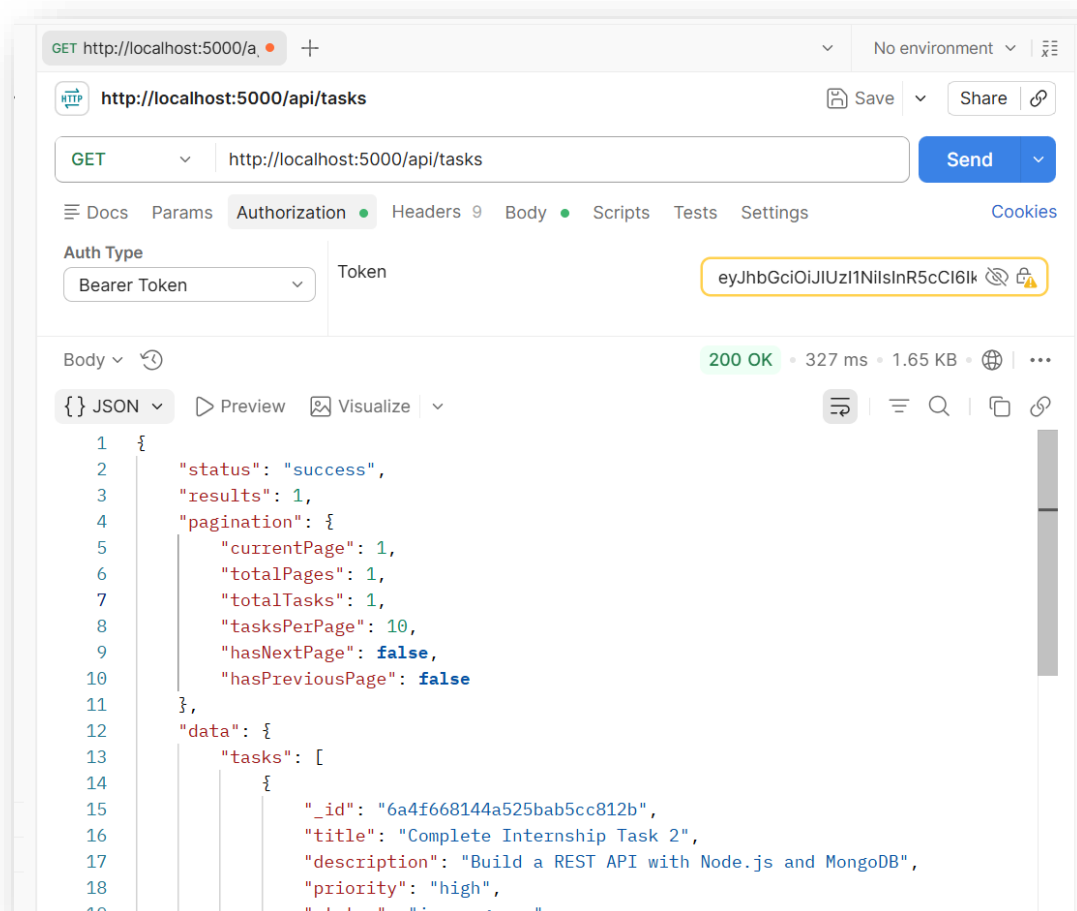
```
GET /api/tasks/stats
```

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

```
// 200 OK
```

```
{
  "status": "success",
  "data": {
    "totalTasks": 18,
    "byStatus": { "pending": 6, "in-progress": 7, "completed": 5 },
    "completionRate": 27.8,
    "overdueCount": 3
  }
}
```

```
GET /api/tasks?search=report&status=in-progress&sort=-priority, dueDate&page=1&limit=10
```



9. Centralized Error Handling

9.1 The AppError Class

`utils/AppError.js` defines a custom Error subclass that carries an explicit HTTP `statusCode` and a boolean `isOperational` flag alongside the standard `message` property. Operational errors — expected failure modes such as 'task not found' or 'invalid credentials' — are thrown as AppError instances with an appropriate status code, allowing the error-handling middleware to distinguish them from unanticipated programming errors (bugs), which are instead logged in full and returned to the client as a generic 500 response to avoid leaking internal implementation details.

9.2 The catchAsync Wrapper

Because Express does not natively forward rejected promises from async route handlers to its error-handling middleware, every async controller function is wrapped with `catchAsync`, a higher-order function that returns a new handler catching any rejected promise and invoking `next(error)` automatically. This removes the need for repetitive try/catch boilerplate across every controller while guaranteeing no asynchronous error can crash the process or hang a request unresolved.

9.3 Centralized Middleware — errorHandler.js

As the final middleware in the Express stack, `errorHandler.js` receives every error forwarded via `next(error)` and normalizes it into a consistent JSON error contract before responding. It applies targeted translation logic for several well-known error categories originating from Mongoose and the JWT library:

Source Error	Detected By	Mapped Status	Client Message Intent
Mongoose CastError	error.name === 'CastError'	400 Bad Request	Signals a malformed identifier (e.g. an invalid ObjectId) supplied in the request path.
Mongoose duplicate key (code 11000)	error.code === 11000	409 Conflict	Signals a uniqueness violation, most commonly a duplicate email during registration.
Mongoose ValidationError	error.name === 'ValidationError'	422 Unprocessable Entity	Aggregates all field-level validation failures into a single, structured error list.
JsonWebTokenError	error.name === 'JsonWebTokenError'	401 Unauthorized	Signals a token with an invalid signature, most often a forged or corrupted token.
TokenExpiredError	error.name === 'TokenExpiredError'	401 Unauthorized	Signals an expired token, prompting the client to re-authenticate.
AppError (operational)	error.isOperational === true	error.statusCode	Passes through the explicit status and message defined at the throw site.
Unhandled / programming error	(fallback branch)	500 Internal Server Error	Returns a generic message to the client while the full stack trace is logged server-side for debugging.

This translation layer ensures that regardless of which internal library or layer produced a failure, API consumers always receive a predictable, well-typed JSON error object of the form { status, message, errors? }, making the API safe to integrate against without requiring clients to understand Mongoose or JWT internals.

Document Control

Attribute	Value
Author	Abdul Haseeb
Institute	ITSimplera Solutions — Full Stack Web Development Internship
Task	Week 2 Task
Repository	GitHub Repo Link

— *End of Document* —